

Generierungsparameter

Einstellungen, ihre Wirkung — und Beispiele, sie zu ändern

Zweck

Dieses Dokument beschreibt die Werte, die das Verhalten eines Laufs bestimmen, und die Stellen, an denen sie sich ändern lassen. Es geht nicht um Modellwahl — die steht im Dokument zur Modellauflösung — sondern um die Stellschrauben, die unabhängig vom Modell gelten.

Wo ein Parameter geändert werden kann

Ein Parameter ist nicht an eine einzige Stelle gebunden. Das Projekt kennt sechs Ebenen, auf denen Werte gesetzt werden können, und sie haben eine feste Rangfolge: Eine höhere Ebene überschreibt eine niedrigere. Wer einen Wert ändert, sollte deshalb wissen, an welcher Ebene er arbeitet — sonst bleibt eine Änderung in der `.env` wirkungslos, ohne dass der Grund sichtbar wird.

Ebene	Wer ändert	Beständigkeit
Argument auf der Kommandozeile	Anwender	nur für diesen einen Lauf
Umgebungsvariable des Prozesses	Betreiber	für den laufenden Prozess
<code>.env</code> im Projektordner	Anwender	dauerhaft, bis die Datei geändert wird
Aufrufparameter im Python-Code	Anwender bzw. eigener Code	nur für diesen Aufruf
Feld der Anwendungsklasse	Anwender bzw. eigener Code	für die Lebensdauer des Objekts
Standardwert in <code>core/config.py</code>	Entwickler	für alle, bis der Code geändert wird

Die Rangfolge

- Argument schlägt Umgebungsvariable, Umgebungsvariable schlägt .env, .env schlägt den Standardwert im Code. Ein Aufrufparameter im Python-Code steht auf derselben Höhe wie ein Argument: Er wird übergeben und nicht gesucht.

Beispiele: Parameter ändern

1. Kommandozeile — Argumente je Lauf

Der Anwender überschreibt Werte je Aufruf, ohne Dateien anzufassen. Das ist der übliche Weg beim Ausprobieren, weil der Lauf danach wieder mit den alten Werten startet.

Befehl	Wirkung
uv run rag --ask "FRAGE" --k 5	fünf Treffer statt drei in den Kontext holen
uv run rag --chunks --chunk-size 1200 --chunk-overlap 150	größere Chunks mit größerer Überlappung
uv run chunking --strategy token --chunk-size 600	nach Token statt nach Zeichen schneiden
uv run chunking --compare	alle Strategien bei gleichem Zielwert vergleichen
uv run rag --ask "FRAGE" --mode hybrid	Stichwortsuche zusätzlich zur Einbettung
uv run rag --ask "FRAGE" --embedding-model nvidia/llama-nemotron-embed-vl-1b-v2	anderes Einbettungsmodell
uv run rag --language en --load	englische Wissensbasis verwenden
uv run rag --load --format txt	Textfassung statt PDF laden
uv run rag --ask "FRAGE" --temperature 0.4	mehr Streuung in der Antwort
uv run rag --ask "FRAGE" --no-progress	Fortschrittszeilen unterdrücken

Die Temperatur kommt auf der Kommandozeile mit Standard 0.0 und überschreibt damit denselben Wert aus den Einstellungen. Das ist beabsichtigt: Der Aufruf soll für sich vollständig bestimmt sein und nicht davon abhängen, was gerade in der .env steht.

Einige Argumente sind auf eine Auswahl begrenzt und werden geprüft, bevor die Anwendung überhaupt startet. --strategy nimmt nur recursive, character oder token an, --mode nur dense oder hybrid, --format nur pdf oder txt, --language nur de oder en. Ein unzulässiger Wert wird abgewiesen, statt später einen Fehler auszulösen.

Auch die Vorverarbeitungs-CLIs haben ihre eigenen Schalter. Die Bereinigung etwa lässt sich so für einen Lauf anders einstellen:

Befehl	Wirkung
uv run cleaning --no-hyphens --steps	Silbentrennung nicht auflösen, Wirksamkeit zeigen
uv run cleaning --keep-empty	leere Dokumente behalten statt verwerfen
uv run loading --preview 400 --encoding latin-1	andere Kodierung, längere Vorschau
uv run python -m rag_project.app.main_chunking --chunk-size 400 --show 0	kleinere Chunks, keine Vorschau

2. Umgebungsvariable des Prozesses

Parameter lassen sich für einen einzelnen Aufruf in der Umgebung setzen. Der Wert gilt nur für diesen Prozess und überschreibt die .env, weil das Laden der .env keine bereits gesetzte Prozessvariable überschreibt.

PowerShell — die Zuweisung vor dem Befehl, getrennt durch ein Semikolon:

Befehl	Wirkung
\$env:TEMPERATURE="0.5"; uv run rag --ask "FRAGE"	Temperatur für diesen Aufruf
\$env:MAX_TOKENS="2048"; uv run rag --ask "FRAGE"	längere Antworten erlauben
\$env:REQUEST_TIMEOUT="120"; uv run rag-project --check	mehr Zeit für langsame Anfragen
\$env:DOCS_DIR="documents"; uv run loading --show-paths	anderer Dokumentenordner
\$env:DEFAULT_DOCUMENT="handbook.txt"; uv run loading	anderes Standarddokument

Befehl	Wirkung
<code>\$env:DOCUMENT_FILES="handbuch.txt,anhang.md"; uv run chunking</code>	mehrere Dateien auf einmal
<code>\$env:TEMPERATURE=""</code>	Wert leeren, Standard greift wieder

Linux und macOS — entweder vorangestellt oder für die Sitzung exportiert:

Befehl	Wirkung
<code>TEMPERATURE="0.5" uv run rag --ask "FRAGE"</code>	nur für diesen einen Aufruf
<code>export MAX_TOKENS="2048"</code>	für die laufende Sitzung setzen
<code>export DOCS_DIR="documents"</code>	Dokumentenordner umstellen
<code>unset MAX_TOKENS</code>	Zurücksetzen auf die .env bzw. den Code-Standard

Warum die Umgebung gewinnt

- In einem Container oder in einer CI-Umgebung gewinnt die echte Umgebung, lokal gewinnt die .env. Ist eine Variable bereits gesetzt, bleibt sie gesetzt — die .env füllt nur, was noch leer ist.

3. .env — dauerhaft für das Projekt

Der dauerhafte Weg: Die Werte in der .env in der Projektwurzel setzen, dort wo auch pyproject.toml liegt. Danach gelten sie für jeden Lauf, bis die Datei wieder geändert wird.

So sieht eine ausgefüllte .env im relevanten Teil aus:

- `CHAT_PROVIDER=openrouter`
- `CHAT_MODEL=openrouter:z-ai/glm-5.3`
- `TEMPERATURE=0.3`
- `MAX_TOKENS=1024`
- `REQUEST_TIMEOUT=60.0`
- `DOCS_DIR=docs`
- `DEFAULT_DOCUMENT=handbuch.txt`
- `DOCUMENT_FILES=`

Einige Zeilen haben eine Eigenheit, die man kennen sollte: CHAT_MODEL und DOCUMENT_FILES dürfen leer bleiben. Ein leerer Wert wirkt wie nicht gesetzt — CHAT_MODEL überlässt die Wahl dem Provider-Standard, DOCUMENT_FILES lässt DEFAULT_DOCUMENT gelten. Wer mehrere Quellen will, trägt sie kommasetrennt ein; dann bestimmt diese Liste die Auswahl.

Variable	Bedeutung	Standard im Code
CHAT_PROVIDER	openrouter oder openai	openrouter
CHAT_MODEL	Modellkennung; leer heißt Standard des Providers	keiner
TEMPERATURE	Streuung der Antwort, 0.0 bis 2.0	0.0
MAX_TOKENS	Obergrenze der Antwortlänge	1024
REQUEST_TIMEOUT	Zeitüberschreitung in Sekunden	60.0
DOCS_DIR	Ordner der Quelldokumente	docs
DEFAULT_DOCUMENT	Standarddatei darin	handbuch.txt
DOCUMENT_FILES	kommasetrennte Auswahl; schlägt DEFAULT_DOCUMENT	leer

Auch die Temperatur wird geprüft: Ein Wert außerhalb von 0.0 bis 2.0 wird beim Erzeugen der Einstellungen abgelehnt, statt später ein unerwartetes Ergebnis zu liefern.

Die Einstellungen werden einmal gelesen

- `get_settings()` erzeugt die Einstellungen genau einmal pro Prozess. Eine Änderung an der `.env` wirkt deshalb nicht in einem laufenden Programm — sie wirkt beim nächsten Start. Beim Chatbot heißt das: Sitzung beenden und neu starten.

4. Python-Code — Aufrufparameter

Wer die Bibliotheken direkt nutzt statt der Kommandozeile, übergibt die Werte als Argumente. Dafür braucht es kein Terminal — ein Skript oder ein Notizbuch reicht, und die Werte gelten nur für diesen Aufruf.

Aufruf	Wirkung
<code>run_pipeline(chunk_size=1200, chunk_overlap=150)</code>	größere Chunks für einen Lauf

Aufruf	Wirkung
<code>run_pipeline(strategy="token", language="en")</code>	Token-Strategie auf der englischen Fassung
<code>split_documents(docs, strategy="character", chunk_size=600)</code>	direkt chunken, ohne CLI
<code>clean_documents(docs, drop_empty=False)</code>	leere Dokumente behalten
<code>clean_documents(docs, join_hyphens=False, unify_bullets=False)</code>	einzelne Schritte abschalten
<code>load_documents(paths, encoding="latin-1")</code>	andere Kodierung beim Laden
<code>build_vectorstore(chunks, embeddings, retrieval_mode=RetrievalMode.HYBRID)</code>	Index mit hybrider Suche
<code>build_rag_chain(store, llm, k=5)</code>	fünf Treffer in den Kontext
<code>build_llm("openrouter:z-ai/glm-5.3", streaming=True, temperature=0.4)</code>	Client mit eigener Temperatur

Ein Sonderfall sind die Bereinigungsfunktionen: `normalize_unicode_text`, `join_hyphens` und `unify_bullets` schalten einzelne Schritte ab. Auf der Kommandozeile heißen dieselben Schalter `--no-unicode`, `--no-hyphens` und `--no-bullets`; im Code übergibt man sie als benannte Argumente. Beide Wege enden in derselben Funktion.

Manche Parameter werden durchgereicht, andere gesetzt. Ein durchgereichter Wert wie `language` wird nur weitergereicht und nicht geprüft; ein gesetzter Wert wie `temperature` landet als Parameter an der Bibliothek und wird dort geprüft. Wer wissen will, welcher Fall vorliegt, schaut, ob die Funktion den Wert nur annimmt oder ob sie ihn selbst verbraucht.

5. Felder der Anwendungsklasse

Anwendungsklassen halten ihre Werte als Felder, die beim Erzeugen gesetzt werden. Sie wirken für die gesamte Lebensdauer des Objekts — anders als ein Aufrufparameter, der nur für einen einzelnen Aufruf gilt.

Aufruf	Wirkung
<code>ChatBot()</code>	Standard-Prompt, Modell aus der <code>.env</code>
<code>ChatBot(model="openrouter:z-ai/glm-5.3")</code>	Modell für die Lebensdauer des Bots
<code>ChatBot(temperature=0.4)</code>	Temperatur des Bots setzen

Aufruf	Wirkung
ChatBot(system_prompt="Antworte nur auf Deutsch.")	eigener System-Prompt
uv run chatbot --no-stream	Instanz mit dem Ausgabemodus invoke

Der ChatBot zeigt den Unterschied zwischen den Ebenen besonders deutlich. Ist das Temperaturfeld None, wird der Parameter gar nicht erst gesetzt, und es gilt, was der Client selbst als Vorgabe hat. Wer die Temperatur ändern will, übergibt sie beim Erzeugen des Bots.

Zur Laufzeit lässt sich der System-Prompt ändern; die REPL bietet dafür /system. Ein neues Modell verlangt dagegen einen Neustart — /model zeigt nur den aktiven Zustand an.

6. Standardwerte im Code

Die unterste Ebene liegt in core/config.py. Dort stehen die Vorgaben, die gelten, wenn niemand etwas anderes setzt: Temperatur 0.0, max_tokens 1024, Zeitüberschreitung 60 Sekunden, Dokumentenordner docs, Standarddokument handbuch.txt, Anbieter openrouter. Diese Werte sind die letzte Instanz der Auflösung.

Ein Sonderfall sind die beiden Anbieteradressen. Sie stehen bewusst als Konstanten im Code und nicht in den Einstellungen: Der Entwickler kann sie ändern, der Anwender nicht. Das ist eine Absicht, keine Auslassung — wer die Adresse ändern will, ändert den Code.

Wer einen Standardwert dauerhaft verschiebt, ändert ihn an dieser Stelle. Das betrifft alle, die das Projekt nutzen, und gehört deshalb in eine bewusste Entscheidung, nicht in eine beiläufige Anpassung.

Was auf welcher Ebene geprüft wird

Nicht jede Ebene prüft ihre Werte gleich streng. Das erklärt, warum ein unzulässiger Wert an einer Stelle sofort auffällt und an einer anderen erst später.

Ebene	Prüfung
Standardwert im Code	keine — er ist die Vorgabe
Kommandozeile	durch die Auswahl der erlaubten Werte im Argumentenparser
Umgebungsvariable	durch die Validierung der Einstellungen beim Erzeugen

Ebene	Prüfung
.env	wie die Umgebungsvariable, weil sie in dieselben Einstellungen mündet
Aufrufparameter im Python-Code	durch die Zusicherungen der aufgerufenen Funktion
Feld der Anwendungsklasse	durch die Zusicherungen des Konstruktors

Ein Wert, der nicht überall gleich wirkt

Die Temperatur ist das beste Beispiel dafür, dass eine Änderung nicht garantiert durchkommt. Sie hängt am Modell, nicht am Wunsch: Nicht jedes Modell erlaubt den Parameter, weil manche Reasoning-Modelle ihn ablehnen. Der Katalog führt die Freigabe je Modell, und die Auflösung setzt den Parameter nur, wenn sie erlaubt ist — sonst bleibt er gar nicht erst im Aufruf.

Wer die Temperatur an irgendeiner der sechs Ebenen ändert, ändert damit also nur einen Wunsch. Ob er zur Anwendung kommt, entscheidet das Modell. Die Ausgabe von `--check` zeigt deshalb `n/a` im Abschnitt Ziel, wenn der Parameter nicht zum Einsatz kommt.

Befehl	Beobachtung
<code>uv run rag-project --check --model openai:gpt-5-pro</code>	Temperatur erscheint als <code>n/a</code> — das Modell erlaubt sie nicht
<code>uv run rag-project --check --model openai:gpt-4o-mini</code>	Temperatur erscheint mit dem gesetzten Wert

Temperatur: worauf der Wert zielt

Die Temperatur steuert, wie stark die Antwort streut. Für RAG ist ein niedriger Wert die richtige Wahl: Die Antwort soll sich auf den mitgelieferten Kontext stützen, nicht auf Formulierungsvarianten ausweichen. Der Standard von 0.0 ist deshalb bewusst gewählt.

Wert	Wirkung
0.0	so deterministisch wie möglich; Standard für RAG
0.2 bis 0.5	leichte Variation, noch kontexttreu
über 1.0	deutliche Streuung; für RAG ungeeignet

Antwortlänge und Zeitüberschreitung

MAX_TOKENS begrenzt die Länge der Antwort. Der Standard von 1024 reicht für kurze, prägnante Antworten, wie sie der RAG-Prompt verlangt. Wird er zu knapp gesetzt, bricht die Antwort mitten im Satz ab; wird er zu großzügig gesetzt, steigen die Kosten, ohne dass die Antwort besser wird.

REQUEST_TIMEOUT begrenzt die Wartezeit auf den Anbieter. Der Standard von 60 Sekunden ist für einzelne Anfragen großzügig; bei einem Modell mit explizitem Nachdenken kann er knapp werden. Er wird am Client gesetzt, nicht am Anbieter.

Chunk-Größe und Überlappung im Zusammenspiel

Die beiden Chunk-Werte wirken zusammen und bestimmen, wie viel Kontext jeder Treffer trägt. Größere Chunks enthalten mehr Zusammenhang, verbrauchen aber mehr Platz im Kontextfenster und verwässern die Treffer: Ein langer Chunk spricht viele Themen an und passt dadurch schlechter zu einer konkreten Frage. Kleinere Chunks treffen genauer, verlieren aber den Zusammenhang um den Fund herum.

Die Überlappung ist die Versicherung gegen Grenzfälle: Ein Satz, der genau auf eine Grenze fällt, wäre ohne sie in beiden Chunks unvollständig. Ein Wert von rund einem Achtel der Zielgröße ist ein üblicher Ausgangspunkt — bei 800 also etwa 100. Beide Werte lassen sich auf allen Ebenen ändern; am häufigsten geschehen sie als Argument oder als Aufrufparameter.

Trefferzahl

Die Trefferzahl bestimmt, wie viele Chunks in den Kontext wandern. Zu wenige Treffer lassen eine beantwortbare Frage unbeantwortet; zu viele füllen den Kontext mit Randthemen und verdrängen die eigentlich passenden Stellen. Drei Treffer sind ein guter Ausgangswert für ein Handbuch mit klaren Abschnitten.

Der Wert liegt als Argument --k und als Aufrufparameter k vor. Auf beiden Wegen gilt er nur für den jeweiligen Lauf beziehungsweise das jeweilige Objekt; in der .env hat er keine Entsprechung.

Einstellungen prüfen

Bevor ein Lauf startet, lässt sich der aktive Zustand anzeigen. Die Schlüssel erscheinen dabei nie im Klartext, sondern nur als Angabe, ob gesetzt und mit welcher Länge.

Befehl	Wirkung
uv run rag-project --show-config	aktive Einstellungen mit Fingerabdrücken
uv run rag-project --check	löst das Ziel auf und zeigt die tatsächlichen Parameter
uv run rag --config	dieselbe Auskunft aus der RAG-Anwendung

Befehl	Wirkung
uv run rag --queries	die Testfragen der eingestellten Sprache

Weitere Umgebungsvariablen

Variable	Zweck
OPENAI_API_KEY	Schlüssel des Anbieters openai
OPENROUTER_API_KEY	Schlüssel des Anbieters openrouter
USER_AGENT	Kennung für abgerufene Netzinhalte
WIKIPEDIA_USER_AGENT	Kennung speziell für Wikipedia-Abrufe

Es wird immer nur der Schlüssel des aktiven Anbieters geprüft. Ein reiner OpenRouter-Nutzer braucht keinen OpenAI-Schlüssel. Erst ein Wechsel des Anbieters — per `CHAT_PROVIDER` oder per Präfix in der Modellkennung — erzwingt die Anwesenheit des anderen Schlüssels.

Kurz: wo ändere ich was?

Ich will ...	Ebene	Beispiel
etwas einmalig ausprobieren	Kommandozeile	uv run rag --ask "FRAGE" --k 5
etwas für einen Prozess setzen	Umgebungsvariable	<code>\$env:TEMPERATURE="0.5"; uv run rag --ask "FRAGE"</code>
etwas dauerhaft für mich einstellen	.env	TEMPERATURE=0.3
etwas in einem eigenen Skript steuern	Python-Aufruf	<code>run_pipeline(chunk_size=1200)</code>
etwas für ein Objekt setzen	Klassenfeld	<code>ChatBot(temperature=0.4)</code>
das Verhalten des Projekts verschieben	Code-Standard	Wert in <code>core/config.py</code> ändern